SDU

Queues
Semaphores
More debugging

Course outline

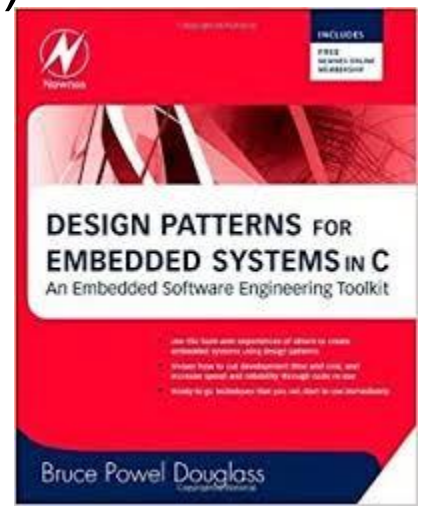
- **The plan is indicative, subject to change!**

Date	Lecture	Subject(s)	Lab	Assignment due
Feb 2	1	Introduction; ARM Cortex-M4	Lab1: Setting up the development env.	
Feb 9	2	EMP coding standard; Bit manipulation	Lab2: Bit manipulation	
Feb 16	3	State machines; The compiler chain; EMP-Board	Lab3: State machines	PF1 – Assignment 1
Feb 23	4	The task model; The pre-processor	Lab4: A clock radio task model	PF1 – Assignment 2
Mar 2	5	Queues and semaphores; Debugging	Lab5: Debug with a serial connection	PF1 – Assignment 3
Mar 9	6	Run to complete scheduler	Lab6: RTCS, Run to compile scheduler	PF1 – Assignment 4
Mar 16	7	More debugging; C: printf()	Lab7: RTCS, Debugger	PF1 – Assignment 5
March 23	8	FreeRTOS	Lab8: FreeRTOS	PF1 – Assignment 6
March 30 April 6		Easter holiday		
Apr 13	9	More queues; Assembler in C	Lab9: FreeRTOS (continued)	PF1 – Assignment 7
Apr 20	10	Re-entrance		PF1 – Assignment 8
Apr 27	11	Work on the final assignment, consultations		
May 4	12	Poster session		PF2- Final Assignment

Additional suggested books

Embedded System Design using TM4C LaunchPad Development Kit

- Available for free trough Texas Instruments (registration needed)
- Topics: interrupts, GPIO, PWM, UART, etc.

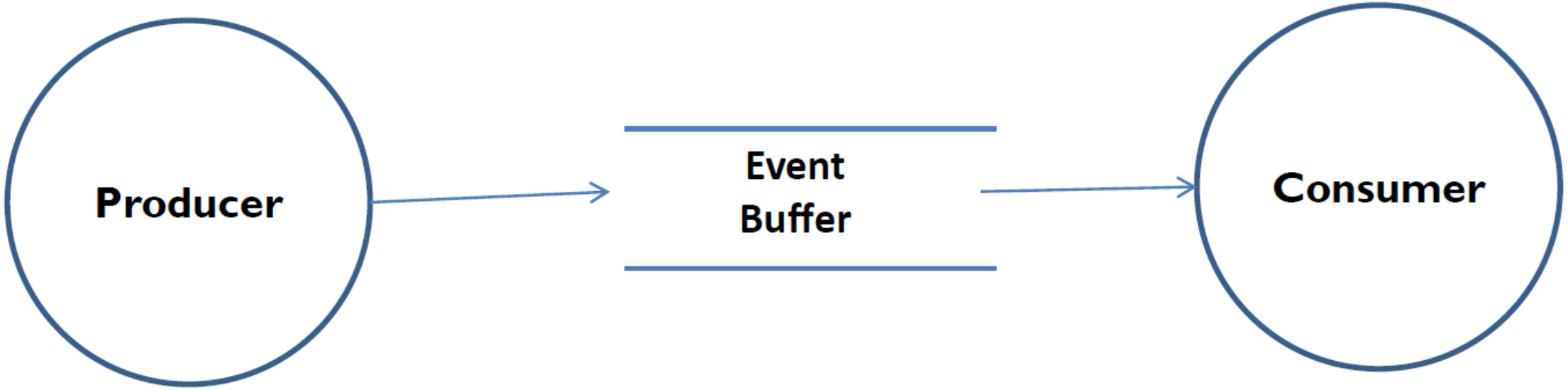


Design Patterns for Embedded Systems in C
Bruce Powel Douglas

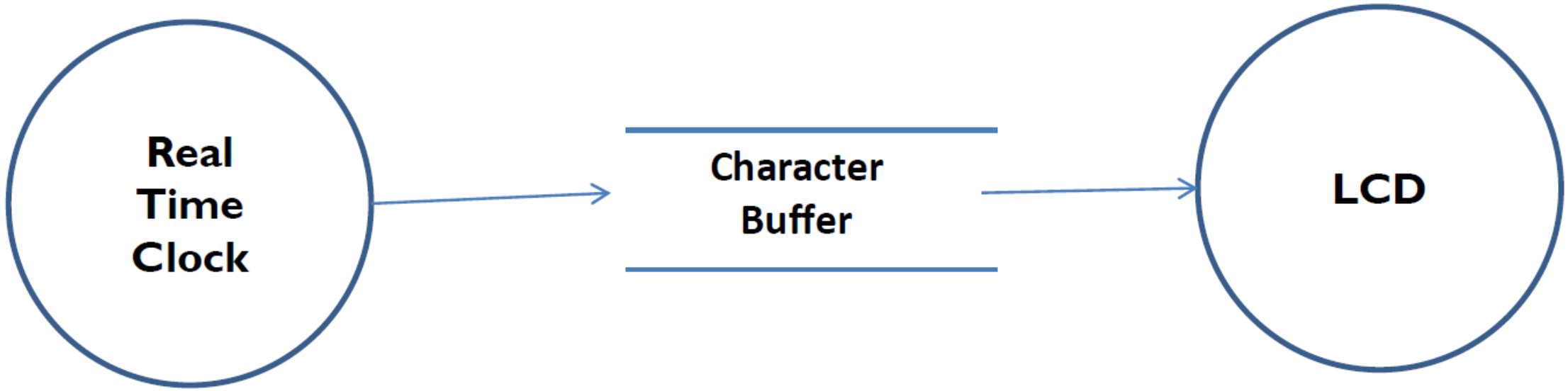
- Not free –based on contents sounds good – but I did not verify
- Interrupt, queuing, state machines, etc.

SDU

Producer – consumer model



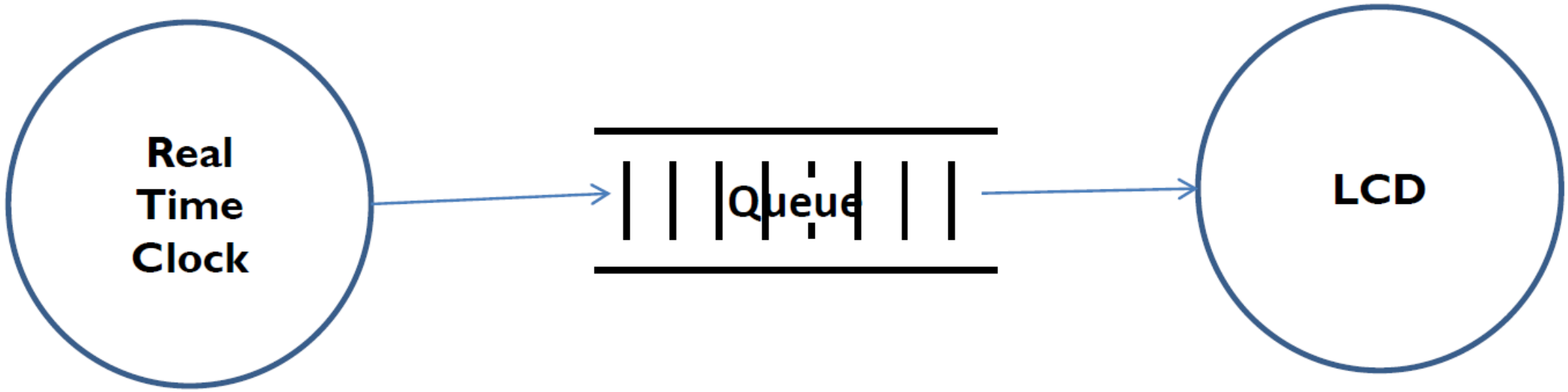
Producer – consumer model



”12:31:55”
100nS/char

”12:31:55”
1mS/char

Producer – consumer model



”12:31:55”
100nS/char

”12:31:55”
1mS/char

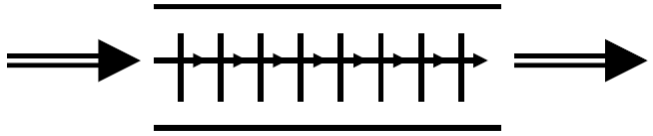
A queue object

A queue is…
…an individual object in the task diagram
…has a unique name, number, id, key or handle
…implement a FIFO strategy – first in first out
…offer an API for:

- creating (Create, Open, …)
- insert data (Put, Write, Send, Post, …)
- fetch data (Get, Read, Receive, Retrieve, …)
- show the state of the queue (number of items in the queue) (Full, Empty, Length, …)
- Maybe: read data without removing it from the queue (Peek, …)

SDU

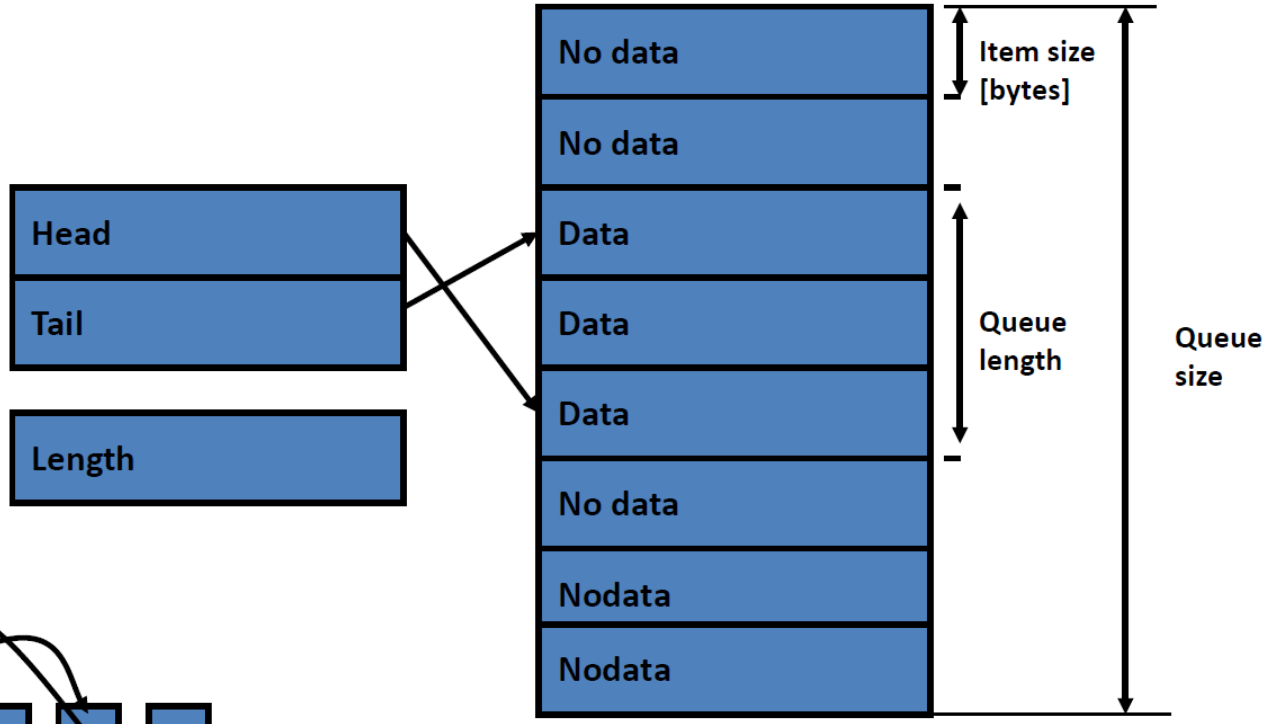
Queue implementation

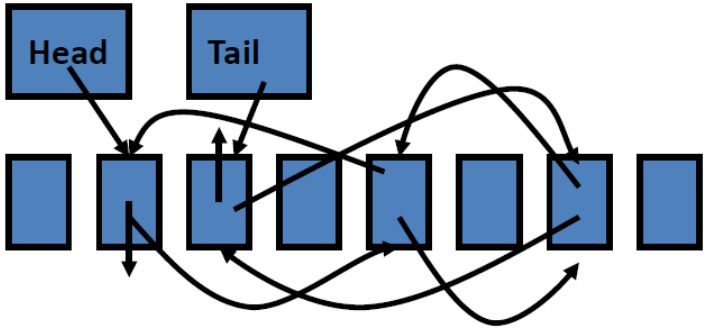


1) Move data through the queue (like supermarket)

2) Use pointers (pull a number -like at the post office)

3) Linked List (like exam – call the next)





Writing to the tail
Reading from the head

SDU

queue.c

```
typedef struct
{
            INT8U      head;
            INT8U      tail;
            INT8U      len;
            INT8U buf[QUEUE_SIZE];
} queue_descriptor;
```

```
queue_descriptor queue_pool[NOF_QUEUES];
```

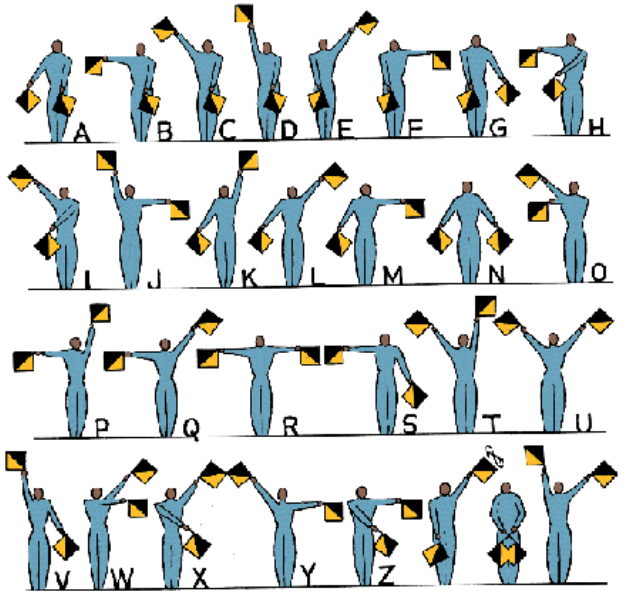
```
INT8U queue_put( id, ch )
INT8U id;
INT8U ch;
/**************************************************
*   Function : See module specification (.h-file).
**************************************************/
{
  INT8U result = FALSE;

  if( id < NOF_QUEUES )
  {
    if( queue_pool[id].len < QUEUE_SIZE )
    {
      queue_pool[id].buf[queue_pool[id].tail++] = ch;
      if( queue_pool[id].tail >= QUEUE_SIZE )
        queue_pool[id].tail = 0;
      queue_pool[id].len++;
      result = TRUE;
    }
  }
  return( result );
}
```

```
INT8U queue_get( id )
INT8U id;
/**************************************************
*   Function : See module specification (.h-file).
**************************************************/
{
  INT8U Ch = '\0';

  if( id < NOF_QUEUES )
  {
    if( queue_pool[id].len )
    {
      Ch = queue_pool[id].buf[queue_pool[id].head++];
      if( queue_pool[id].head >= QUEUE_SIZE )
        queue_pool[id].head = 0;
      queue_pool[id].len--;
    }
  }
  return( Ch );
}
```

Semaphores

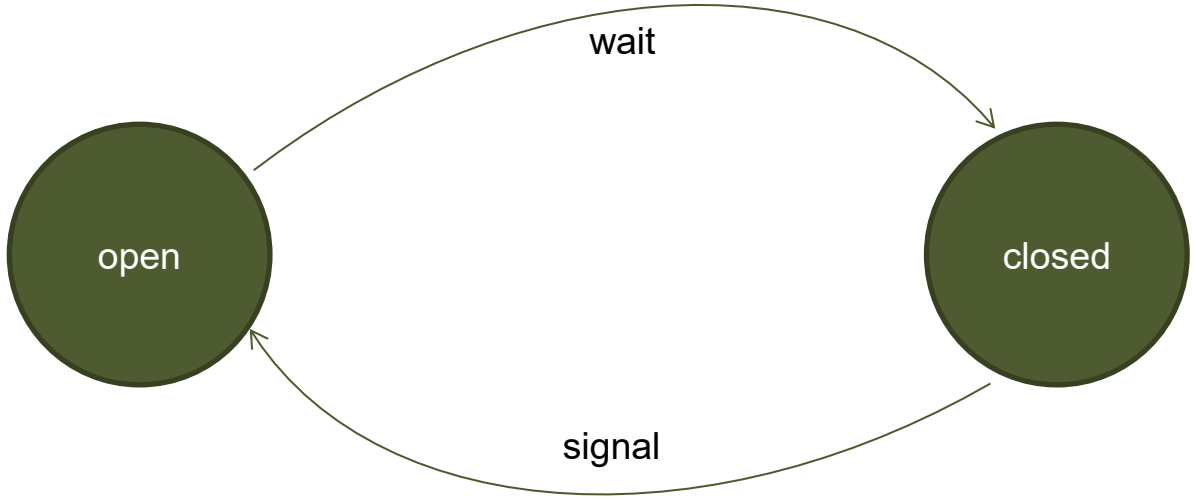


- In real life: visual signaling with arms & flags
- In software: data structure for synchronization
- It's an integer, but
  - The only allowed operations are increment and decrement by one
  - When a task decrements the semaphore, if the result is negative, the task blocks itself and cannot continue until another task increments the semaphore
  - When a task increments the semaphore, if there are other tasks waiting, one of the waiting tasks gets unblocked.
- Counting or binary
- Example parking access control

Semaphore functions

Increment() ⇔ signal() ⇔ V()
Decrement() ⇔ wait() ⇔ P()

increment_and_wake_a_waiting_process_if_any ()
decrement_and_block_if_the_result_is_negative ()



SDU

Why semaphores?

- Easier to understand in code, more intuitive
- Less bugs
- Portable

Semaphore purposes

- Signaling - serialization
- Mutex – mutual exclusion

Semaphore - signaling

- Making sure that statements in different tasks/threads execute in a specific order - serialization

Task A

```
Statement a1
Semaphore.signal()
```

Task B

```
Semaphore.wait()
Statement b1
```

SDU

Semaphore - mutex

- Secures mutually exclusive blocks to prevent concurrent access to the same resource
- Like a binary semaphore
  - Main difference: mutex can only be released by the task which took it while binary semaphores can be released by any task
- Creates critical sections

Task A – clock update

```
mutex.wait()
        #critical section
        update(clock)
mutex.signal()
```

Task B – clock display

```
mutex.wait()
        #critical section
        display(clock)
mutex.signal()
```

Example – contentious meetings with duckie

SDU

Semaphores – more information

The Little Book of Semaphores Allen B. Downey

http://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf

SDU

Semaphores – code

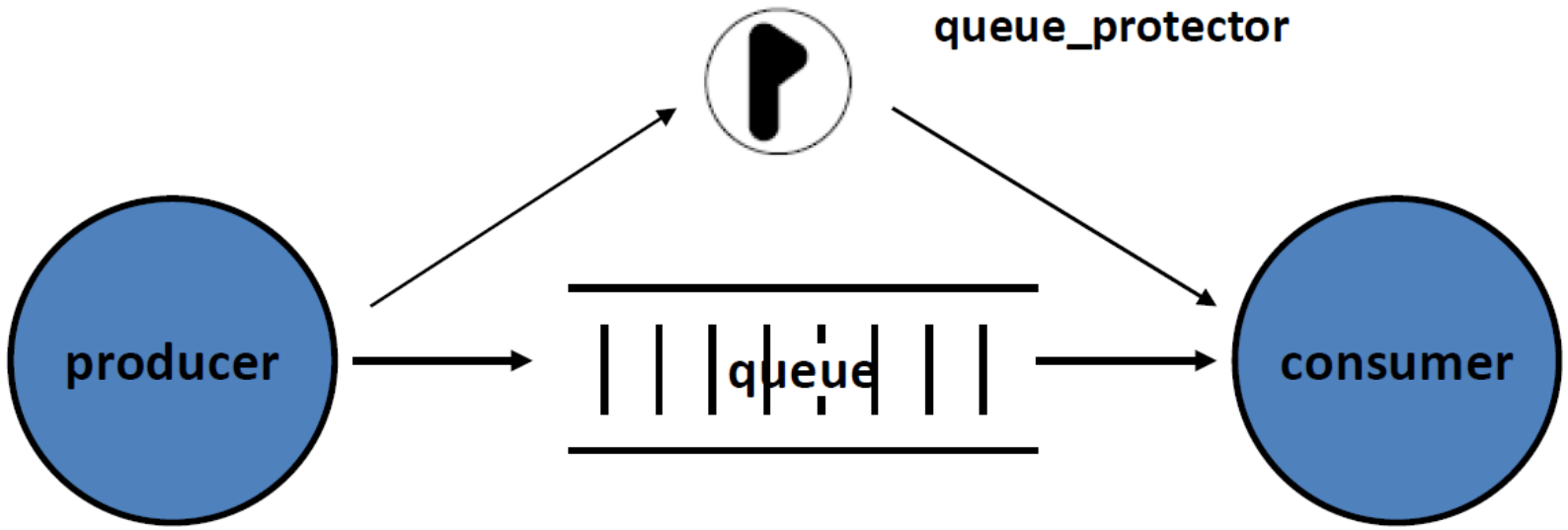
```
BOOLEAN wait( INT8U sem_no)
{
        INT8U Result;

        if( sem_no< MAX_SEM )
        {
                Result = semaphore[sem_no];
                semaphore[sem_no] = FALSE;
        }
        else
                Result = FALSE;
        return( Result );
}

void signal( INT8U sem_no)
{
        if( sem_no< MAX_SEM )
                semaphore[sem_no] = TRUE;
}
```

Protection of queues

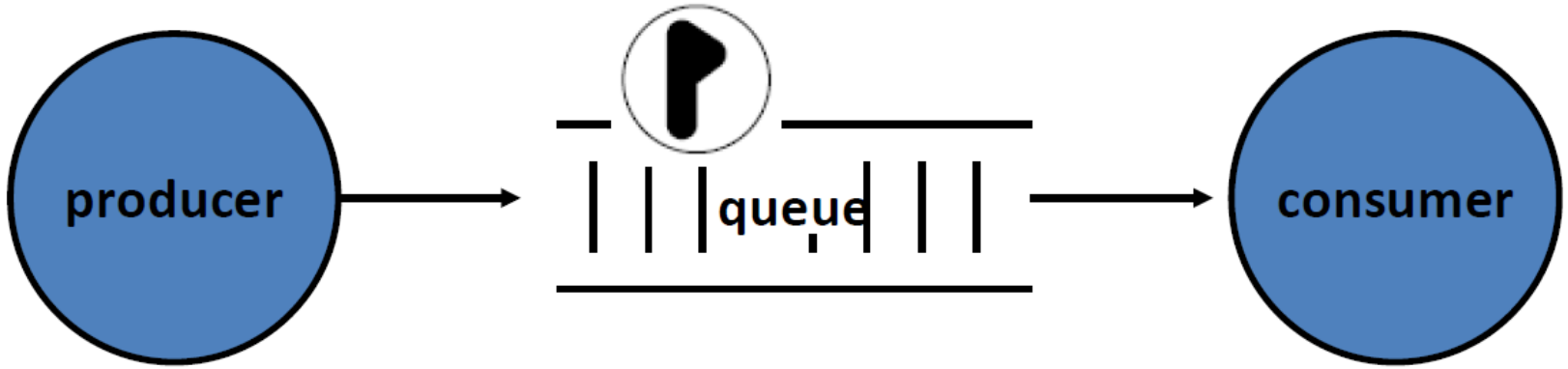
A queue is a shared resource, and as such, it must be protected by a mutex semaphore.



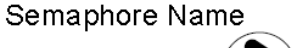
SDU

Protection of queues

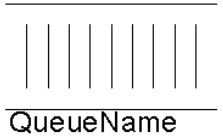
Because of that, the semaphore is often built into the queue or closely connected to the queue.



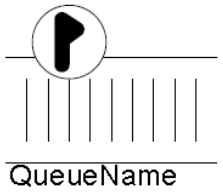
Queue and semaphore symbols in task diagrams



Semaphore and Mutex

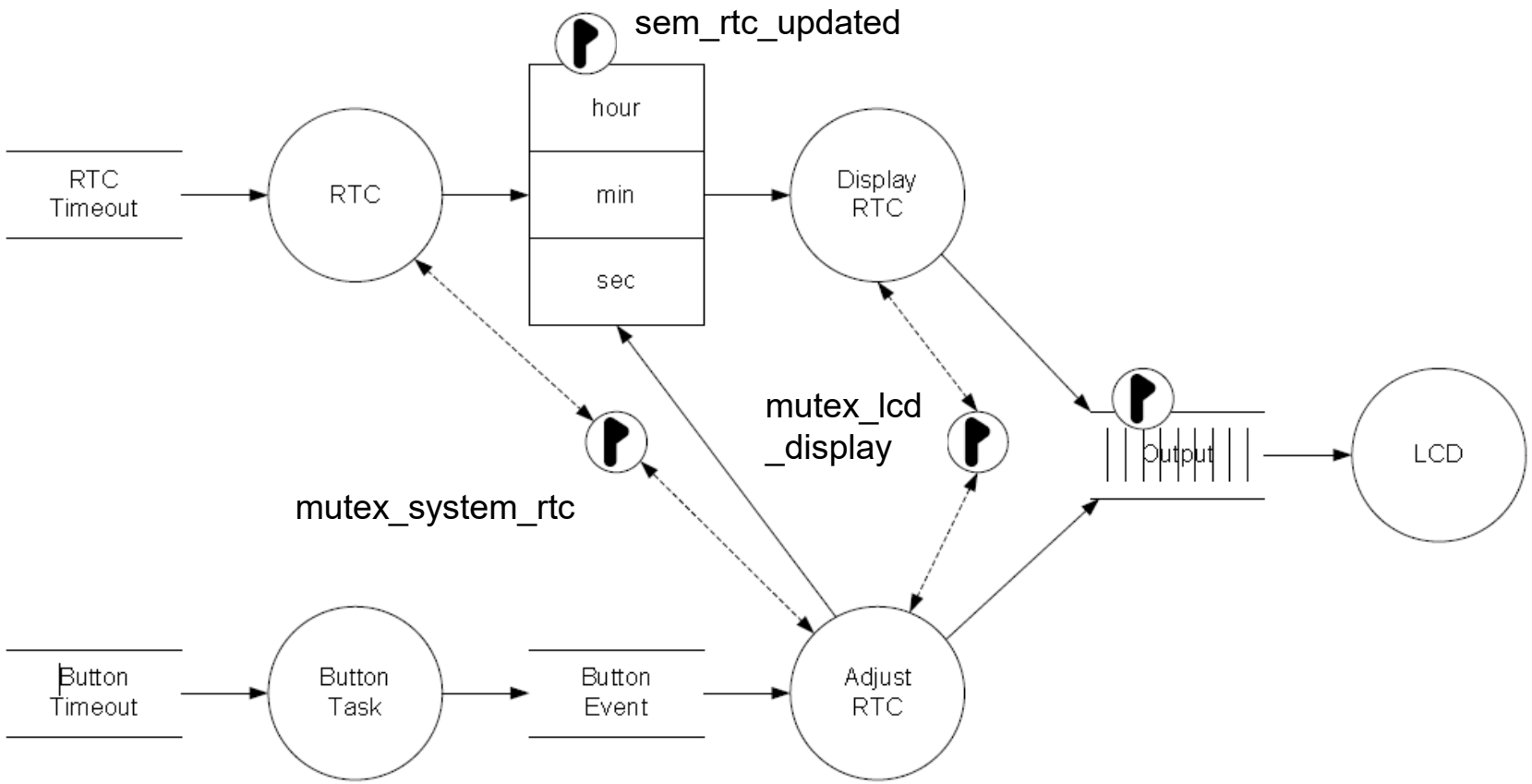


Queue (FIFO)



Queue, protected by Mutex.

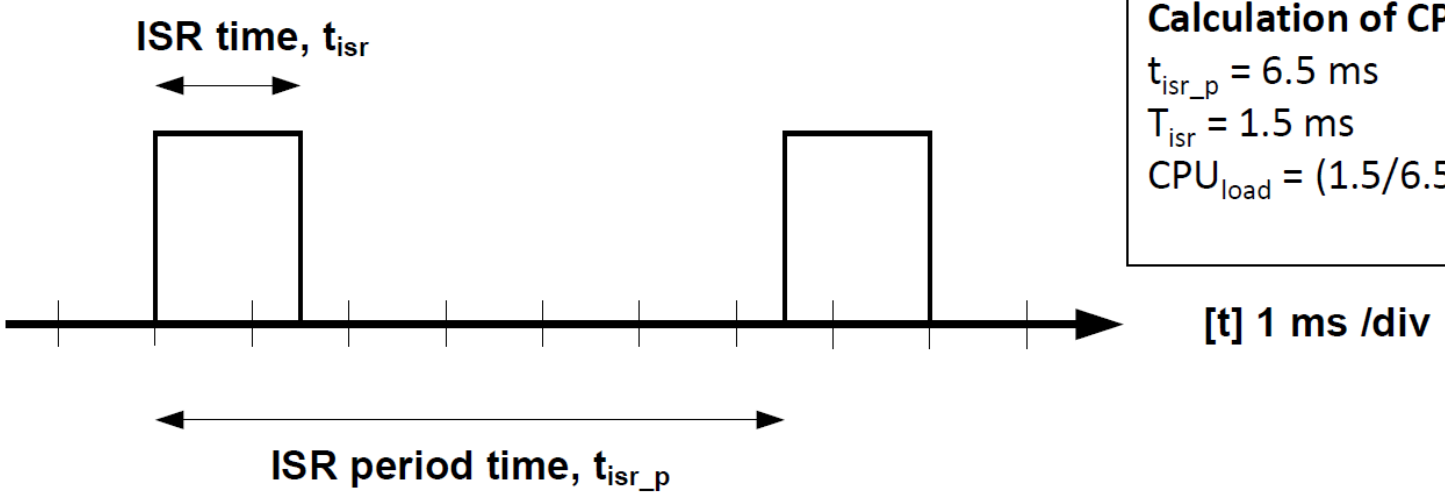
Assignment03 task model solution



Debugging

Pin toggling

- Simplest: toggle LED pin to check execution (alive_timer)
- Measurement of executing time.
- Measure time between events
- Determine if the program has been in a certain function / Interrupt service routine (ISR).
- Oscilloscope / LED's / multimeter.



Calculation of CPU load

t_{isr_p} = 6.5 ms

T_{isr} = 1.5 ms

CPU_{load} = (1.5/6.5)*100 = 23 %

Assert macro

- If the argument value is zero, assert can print a message with line number and source file to a terminal or a display and then stop the program. (implementation dependent)

```
switch (test_var)
{
  case 0:
    ...
    break;

  case        1:
    ...
    break;

  default:
    // Error !
    assert(!(test_var > 1));
}
```


SDU

Debugging using a serial port

- More intelligent debug messages compared to pin toggling.
- Write messages to a terminal or a display
- Uses extra ram and program memory for the printf routine.

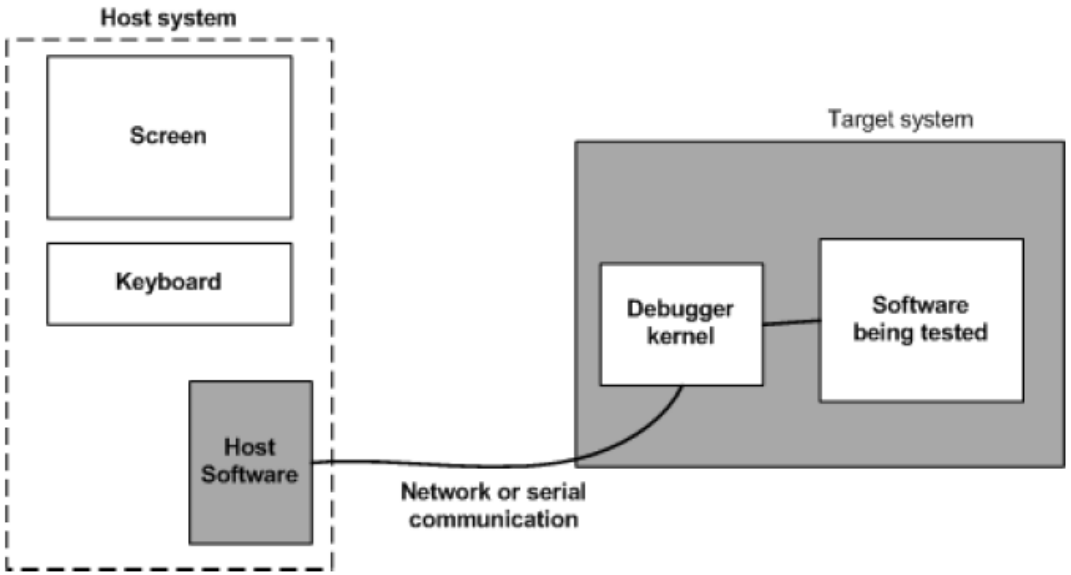
Debuggers/ Tools

- PC simulator
  - Can be used for non-real-time testing.
  - You don’t need hardware.
  - Good for testing hardware independent code

- Rom Monitors
- ICE (In-Circuit Emulators)
- ICD (In-Circuit Debugger) or OCD (On-Chip-debugger

ROM monitor.

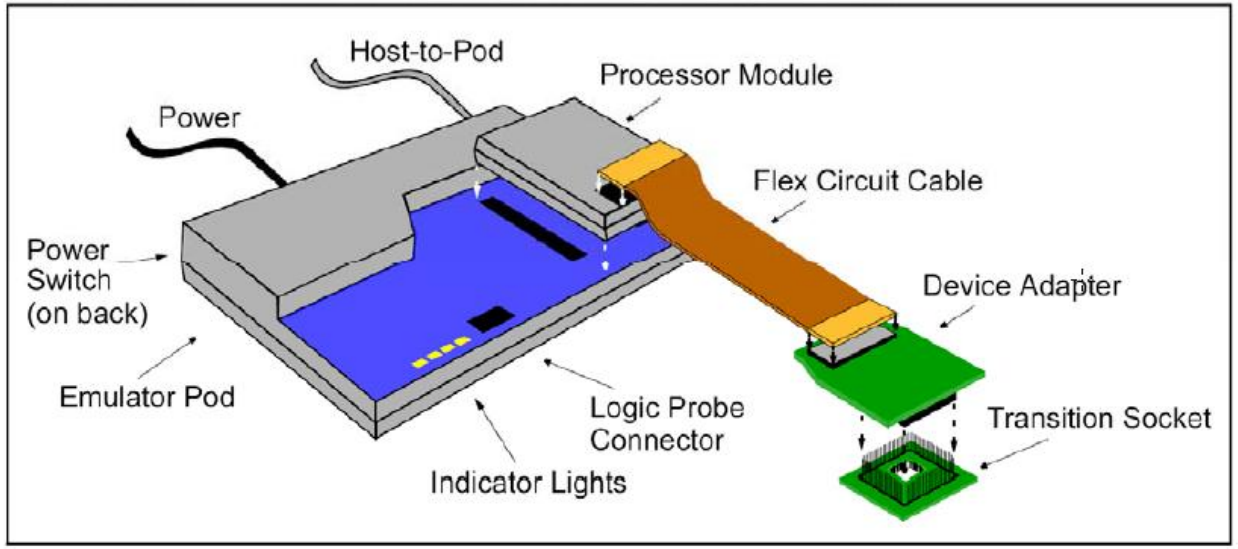
- A cheap solution.
- Uses RAM, ROM and execution time.
- Could be problematic when debugging ISRs
- Connection through serial port



ICE – In-Circuit Emulators

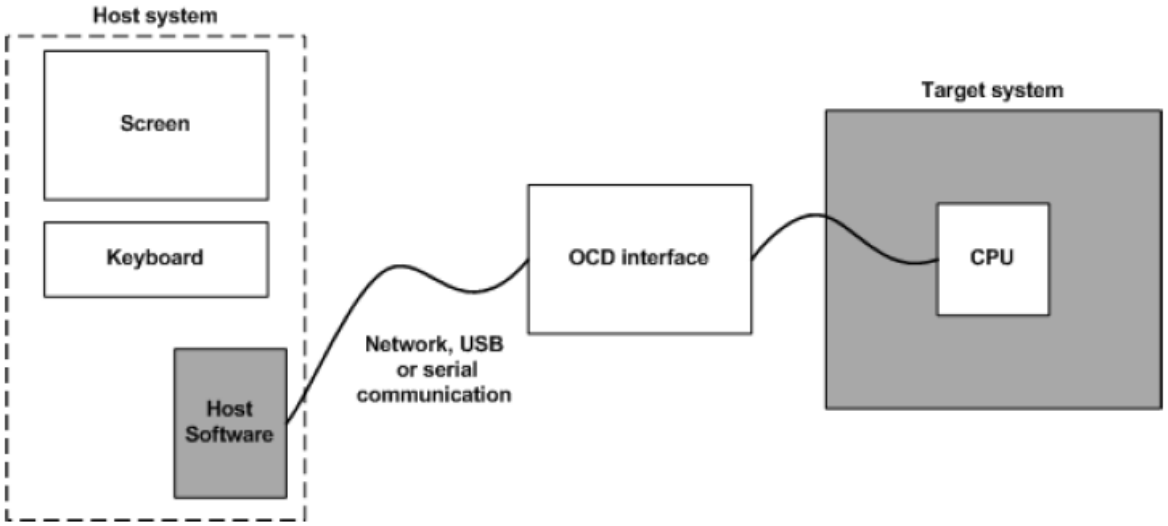
- Replaces the microprocessor on the board.
- Problematic at high frequencies. (Because of the cable)
- Problematic with fine pitch and BGA circuits.
- Mostly used with 8- and 16 bit processors.
- Expensive. (>20 kKr.)

FIGURE 2-1: MPLAB ICE 4000 EMULATOR SYSTEM

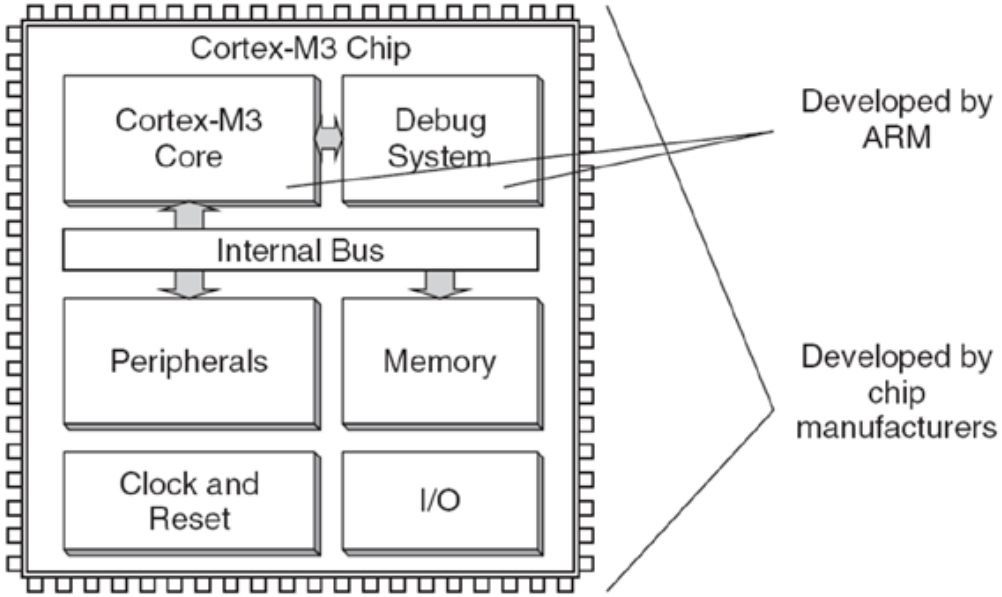


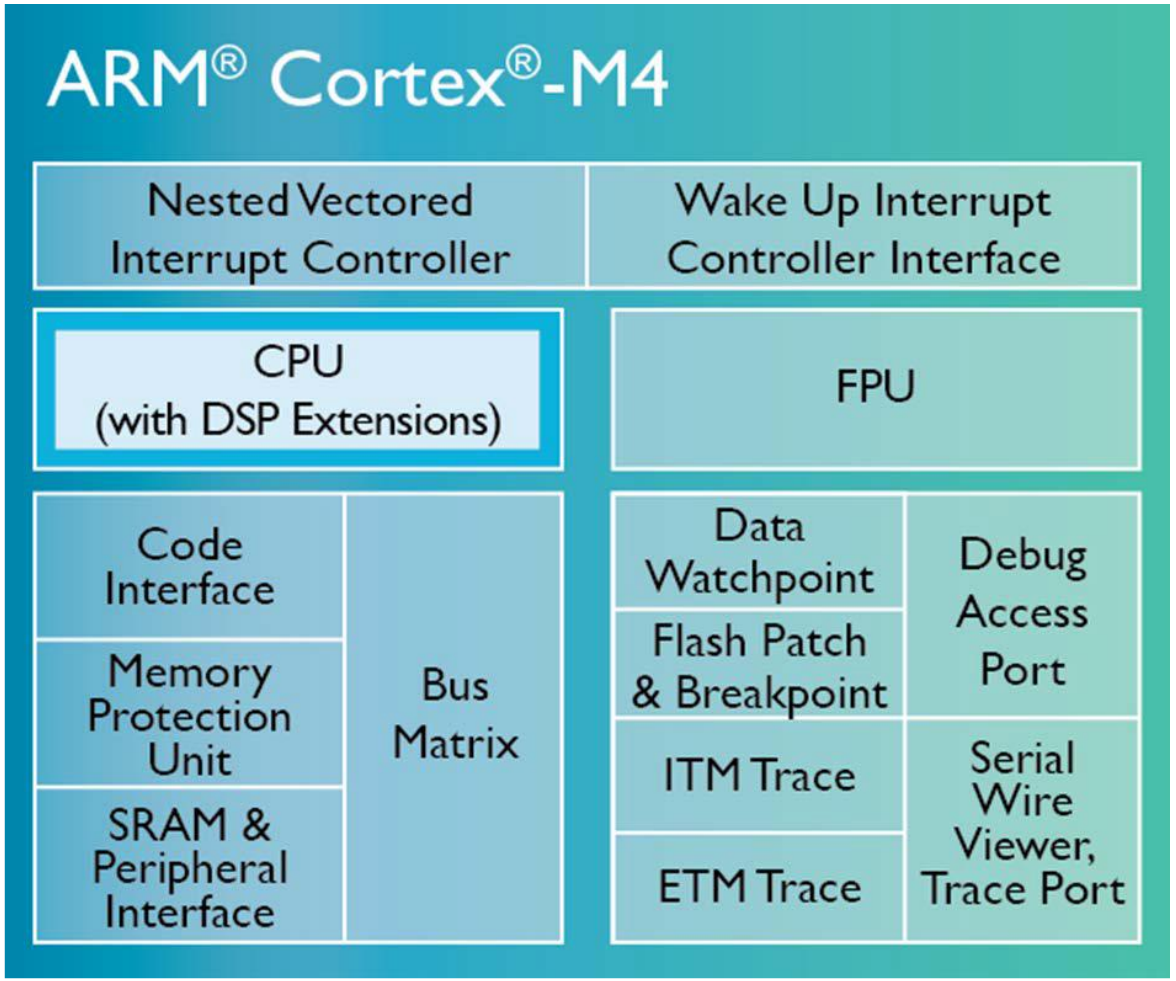
ICD (In-Circuit Debugger) or OCD (On-Chip-debugger)

- Debug HW kernel,build into the CPU.
- Consumes no RAMor ROM.
- Hardware interface: 4-10 pins
  - JTAG(Joint TestAction Group), IEEEstandard
  - BDM(BackgroundDebug Monitor), Motorola



ARM Cortex M3, M4





In Code Composer – already discussed

2017emp - CCS Debug - Ass03b/main.c - Code Composer Studio

File Edit View Project Tools Run Scripts Window Help

Quick Access

Debug

Ass03b [Code Composer Studio - Device Debugging]
- Stellaris In-Circuit Debug Interface/CORTEX_M4_0 (Suspended - HW Breakpoint)
  - main() at main.c:21 0x00000854
  - ResetISR() at tm4c123gh6pm_startup_ccs_gcc.c:298 0x000011D2
  - 0x00000000 (no symbols are defined for 0x00000000)

emp_type.h | main.c | systick.c | systick.h | lcd.c | tmodel.h | sem.c | rtc.c | lcd.h

```
1 #include <stdint.h>
2 #include "tm4c123gh6pm.h"
3 #include "emp_type.h"
4 #include "events.h"
5 #include "gpio.h"
6 #include "systick.h"
7 #include "swtimers.h"
8 #include "tmodel.h"
9 #include "messages.h"
10 #include "systick.h"
11
12
13 INT16S alive_timer = MILLISEC(500);
```

Console

Ass03b

```
CORTEX_M4_0: GEL Output:
Memory Map Initialization Complete
```

LE

SDU